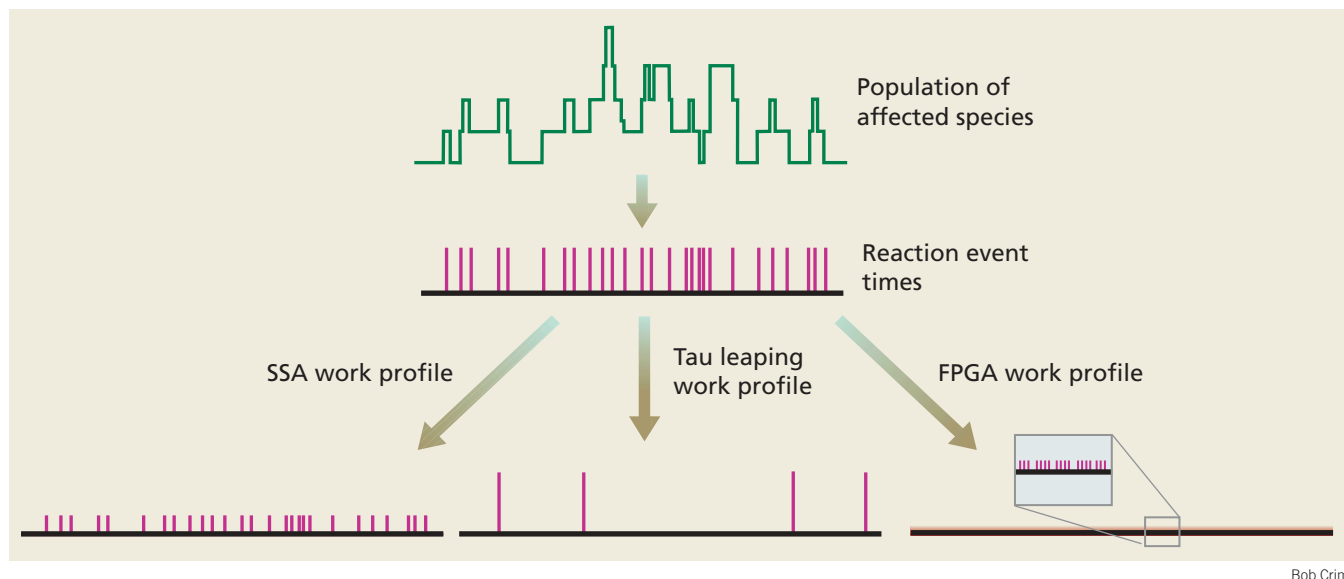


The need for speed in stochastic simulation

Larry Lok

Flexible parallel-processing computer chip technology speeds up stochastic simulation of cellular biochemical systems.



Bob Crimi

Figure 1 An 'iconic' plot of workloads for three stochastic simulation techniques. The horizontal axis is simulation time, but neither time intervals nor work profiles are to scale. The SSA does a moderate amount of work for each reaction event. Tau leaping has larger 'work pulses,' but they encompass many reaction events. Salwinski and Eisenberg's FPGA method slices simulation time finely, but the FPGA's parallel hardware executes each time slice in a single clock cycle.

Cellular biochemical systems have an important stochastic (random) component that affects cells at the phenotypic level¹. The quantitative study of cellular processes with significant random elements has been enabled mainly by stochastic simulation, which accounts for random biochemical events and their consequences. But in spite of intensive development, stochastic simulation software and algorithms have been more or less 'speed challenged' from the beginning. This fact is mainly due to a basic difference between the parallel way in which chemical reactions happen in cells and the way reactions are simulated using conventional uni- and multi-processor computers and clusters. In this issue, Salwinski and Eisenberg² use a field programmable gate array (FPGA), a flexible parallel-processing computer chip, to speed things up by making the simulation operate

a little more like the biochemical reactions that it models.

Gillespie's stochastic simulation algorithm³ (or SSA), the original technique for stochastic biochemical simulation, reveals the main reason for slow simulation speeds: simulating reaction events with mathematical precision requires processing each and every reaction event that occurs between molecules. The processing of an individual reaction event is not hugely complex in the SSA, but neither is it trivial. **Figure 1** gives an 'iconic' picture of computational activity in the SSA: a moderate spike whenever molecules react. The catch is that there might be, well, 'zillions' of reaction events in a single simulated second if reaction rates are high or there are lots of reactant molecules. The rate of reaction events thus becomes the basic speed limit for the SSA. With fast reactions and many reactant molecules, the SSA will run slowly even when the system is at equilibrium.

One acceleration strategy is to abandon absolute mathematical precision in favor of a good-enough approximation. Gillespie has also been a pioneer in this effort. One of

his strategies is called 'tau-leaping'⁴. Instead of processing each reaction event, it moves forward in time by 'leaps' that include many reaction events. The kinds of reaction events that occur, are approximated using algorithms from the mathematical compendium for solving ordinary differential equations. The computational cost of a leap is much higher than a single reaction event under the SSA. Its activity spikes would actually be much taller and farther apart than in the iconic illustration. The speed limit in tau-leaping is connected with how fast the rates of reaction events are changing, rather than with how high the rates are. If the biochemical system is at equilibrium, then tau-leaping zips along because, even though reaction events may be very frequent, they are happening at constant rates.

Salwinski and Eisenberg approach acceleration almost orthogonally: they make processing individual reaction events much faster by accepting some approximation. They do this by using an FPGA, a microchip with many individually reconfigurable units combining processing logic with

Larry Lok is at The Molecular Sciences Institute, 2168 Shattuck Avenue, Berkeley, California 94704, USA.
e-mail: lok@molsci.org

memory. 'Reconfiguration' changes the chip's functionality on the fly at the hardware level, and Salwinski and Eisenberg configure the chip's hardware to do stochastic simulations. For example, every unit gets its own pseudo random number generator, which runs as fast as if it had been built in: it can generate a random number in a single clock cycle. Software alone just can't do that. A reaction is assigned to each chip unit, and its job on every clock cycle is to decide, by 'rolling the dice' with its random number generator, whether or not one of the unit's reaction events should happen in that cycle. If it should, then the unit makes the characteristic (stoichiometric) changes in molecular populations that correspond with that reaction event. Potentially, a reaction event could happen every clock cycle. Reaction event times have a resolution equal to the simulation time of one clock cycle, which is a reasonable approximation. By comparison, processing a single reaction event *precisely* with the SSA takes a CPU many thousands of its clock cycles.

The tiny blips of activity in the FPGA's illustrated iconic workload (Fig. 1) are much

taller and farther apart than in reality. Their spacing, the length of FPGA's clock cycle in simulation time, brings us to an issue unique to this approach: the relationship between the FPGA clock cycle and the rates of reactions. For the functional units of the FPGA to not interfere with one another, the length of the clock cycle in simulation time must be small enough so that the probability of more than one event in a cycle is negligible. In particular, the FPGA clock cycle must simulate a time interval smaller than the typical time interval between events. If reactions are fast, the FPGA simulation must proceed more slowly through simulated time. This could be a practical problem because it is hard to predict how fast reaction events might happen at some time during a simulation without carrying it out. A refinement of this method might adjust the duration of an FPGA clock cycle as needed throughout the simulation, analogous to an adaptive ordinary differential equation solution.

Will FPGAs 'take over' stochastic simulation of biochemical reaction systems? In the commercial realm of simulating

microchip logic, hardware acceleration was introduced fairly early. When FPGAs became available, they were eagerly adopted for digital simulation. Speed through parallelism was also the motivation then, even though microchip simulation is deterministic, and therefore different from Salwinski and Eisenberg's stochastic simulation setup. Some commercial 'turnkey' digital simulation systems are essentially a workstation with a thick cable running to an FPGA 'sidecar' machine. Download the chip design (read biochemical reaction network) to the sidecar, and you're off and running. In the market, software solutions have great resiliency; they're cheaper, generally more convenient and remain dominant in microchip simulation. But an absolute speed advantage, reliably realized, will always have plenty of enthusiastic customers.

1. Elowitz, M. *et al. Science* **297**, 1183–1186 (2002).
2. Salwinski, L. & Eisenberg, D. *Nat. Biotechnol.* **22**, 1017–1019 (2004).
3. Gillespie, D. J. *Comput. Phys.* **22**, 403–434 (1976).
4. Gillespie, D. J. *Chem. Phys.* **115**, 1716–1733 (2001).